SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF ENGINEERING AND TECHNOLOGY

VADAPALANI – 600 026



E-COMMERCE ENTERPRISE APPLICATION USING SPRING BOOT, HIBERNATE AND NEON SERVERLESS POSTGRESQL

A PROJECT REPORT

Submitted by

PRANESH M S [RA2511003040005]

AVINASH A S [RA2511003040074]

KABIL SAMY K M [RA2511003040084]

DINESH KUMAR [RA2511003040106]

ACADEMIC YEAR

2025-2026



SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
Deemed to be University u/s 3 of UGC Act, 1956
VADAPALANI

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

Certified that 21CSC203P Advanced Programming Practice titled **"E-COMMERCE ENTERPRISE APPLICATION USING SPRING BOOT, HIBERNATE AND NEON SERVERLESS POSTGRESQL"** is a bonafide work of **"PRANESH M S [RA2511003040005], AVINASH A S [RA2511003040074], KABIL SAMY K M [RA2511003040084], DINESH KUMAR [RA2511003040106]"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE **SIGNATURE**

STAFF IN-CHARGE **HEAD OF THE DEPARTMENT**

TABLE OF CONTENTS

CONTENT	PAGE NO
1. INTRODUCTION 1.1 PROJECT AIM AND OBJECTIVES 1.2 BACKGROUND OF PROJECT	**2**
2. SYSTEM ANALYSIS 2.1 SOFTWARE REQUIREMENT SPECIFICATION 2.1.1 GENERAL DESCRIPTION 2.1.2 SYSTEM OBJECTIVES 2.1.3 SYSTEM REQUIREMENTS 2.2 SOFTWARE TOOLS USED	**4**
3. SYSTEM DESIGN 3.1 DATABASE SCREENSHOTS 3.2 DATA FLOW DIAGRAMS & USE CASE	**8**
4. SYSTEM IMPLEMENTATION 4.1 SCREEN SHOTS OF OUTPUT 4.2 SCREENSHOTS OF CODING PART	**13**
5. SYSTEM TESTING 5.1 FUNCTIONALITY TESTING 5.2 EXCEPTION TESTING WITHIN FUNCTIONAL TESTING 5.3 EXAMPLES (TEST CASES MATRIX)	**25**
6. CONCLUSION	**30**
7. REFERENCES	**32**

ABSTRACT:

The E-Commerce Enterprise Web Application is a production-oriented, scalable digital commercial platform engineered utilizing Spring Boot 2.6.4, Hibernate ORM 5.6.5, and Neon Serverless PostgreSQL. Designed to eliminate the operational limitations of conventional offline stores and fragmented inventory systems, the application delivers an integrated e-commerce ecosystem catering to both public consumers and system administrators.

The architectural foundation follows the Model-View-Controller (MVC) paradigm, ensuring modular separation of concerns. The presentation layer utilizes high-performance JavaServer Pages (JSP) paired with Twitter Bootstrap 4 to render responsive, cross-platform user interfaces across desktop, tablet, and mobile browsers. Business logic is orchestrated through Spring Boot controllers and transaction-managed service layers, while the persistence tier utilizes a customized Hibernate SessionFactory connected securely to a cloud-hosted PostgreSQL database on Neon via SSL encryption.

Role-Based Access Control (RBAC) is enforced through Spring Security, establishing clear operational boundaries between customer activities (user registration, catalog discovery, shopping cart aggregation, and instant checkouts) and administrative governance (category orchestration, real-time product updates, and customer directory auditing).

By migrating the relational database tier to Neon's cloud-native serverless architecture, the system guarantees elastic compute scaling, automated connection pooling, and continuous high availability without the overhead of maintaining local database infrastructure. This comprehensive project report documents the entire system engineering lifecycle, encompassing requirement specifications, relational database schema models, architectural diagrams, output screenshots, code implementations, testing matrices, and future expansion roadmaps.

CHAPTER 1
INTRODUCTION

This chapter presents a comprehensive introduction to the E-Commerce Enterprise Web Application project. It discusses the core motivation, project aim, functional objectives, and technical background of the system, establishing the architectural context in which modern digital commercial portals operate.

1.1 PROJECT AIM AND OBJECTIVES

Aim: The primary aim of the E-Commerce Enterprise Web Application project is to engineer a secure, highly responsive, and fault-tolerant electronic marketplace that streamlines product catalog discovery, shopping cart management, and administrative inventory governance. The system leverages enterprise Java technologies and serverless cloud databases to deliver dependable electronic commerce operations.

Objectives:

- **Dynamic Catalog & Storefront:** To design an interactive, accessible storefront interface enabling customers to discover categorized goods, inspect item details, and execute instant purchases seamlessly.
- **Role-Based Access Control (RBAC):** To implement robust security boundaries separating standard customer functions from administrative controls, preventing privilege escalation.
- **Administrative Management Hub:** To construct a unified administrative dashboard for maintaining product categories, adding new products, modifying prices, and auditing user directories.
- **Custom Hibernate SessionFactory:** To configure a dedicated Hibernate ORM persistence layer with explicit SessionFactory session management and transaction demarcation.
- **Cloud Serverless Data Persistence:** To integrate Neon Serverless PostgreSQL with SSL encryption, ensuring high availability, automatic connection pooling, and cloud resilience.

1.2 BACKGROUND OF PROJECT

- In today's globalized digital economy, electronic commerce has transformed from an auxiliary sales channel into the central pillar of retail commerce. Consumers expect uninterrupted, instantaneous access to comprehensive product inventories, transparent pricing structures, and intuitive checkout experiences accessible from any device. Legacy brick-and-mortar storefronts and standalone desktop bookkeeping tools suffer from physical geographic limitations, manual accounting errors, inventory desynchronization, and high operational maintenance overhead.
- The evolution of modern enterprise web frameworks has established server-rendered MVC architectures as the industry standard for robust commercial web applications. The Spring Boot ecosystem provides comprehensive dependency injection, automated transaction management, embedded servlet containers, and declarative security configurations, significantly accelerating production deployment.
- Concurrently, traditional on-premise relational database hosting has encountered severe challenges involving manual capacity provisioning, connection pool saturation, and local hardware single points of failure. The emergence of cloud-native serverless databases—such as Neon Serverless PostgreSQL —resolves these bottlenecks by separating storage from compute, offering instant auto-scaling, automated branching, and pooled SSL-encrypted connections.
- The E-Commerce Enterprise Web Application project unites these advanced enterprise standards. By bridging Spring Boot 2.6.4, Hibernate ORM 5.6.5, and Neon Serverless PostgreSQL, the application provides a resilient, full-stack digital marketplace that models real-world enterprise software engineering.

CHAPTER 2
SYSTEM ANALYSIS

2.1 SOFTWARE REQUIREMENT SPECIFICATION

2.1.1 GENERAL DESCRIPTION:

- The E-Commerce Enterprise Web Application is a web-based commercial shopping and inventory management platform designed to automate retail transactions. It allows consumers to register secure profiles, explore categorized inventories (Fruits, Vegetables, Bakery, Dairy, Meat, Drinks), examine product descriptions and stock counts, and execute cart checkout transactions in real time.
- The system automates retail inventory operations through a centralized administrative portal. Authorized administrators can create new classification categories, add product listings with image specifications, update inventory stock levels, adjust pricing, and review registered customer accounts. Visual alerts and dynamic modals provide immediate operational feedback.
- The platform is engineered to run seamlessly across all standard web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari). HTTP request dispatching is handled by an embedded Apache Tomcat 9 servlet container, while persistent records are securely stored within Neon Serverless PostgreSQL hosted on AWS cloud infrastructure.
- System security is enforced through Spring Security authentication filters and BCrypt password encryption, safeguarding user credentials, preventing unauthorized access to administrative endpoints, and maintaining transactional consistency across all database operations.

2.1.2 SYSTEM OBJECTIVES

The central objective of the E-Commerce Enterprise Web Application is to establish a unified, secure, and responsive platform that coordinates commercial interactions between consumers and catalog administrators. The system is designed to eliminate operational bottlenecks, ensure data integrity, and guarantee uninterrupted availability.

Specific Objectives:

- **1. Automated Product Catalog Management:**
  Simplify the ingestion, updating, and taxonomy categorization of commercial goods via dynamic administrative interfaces, eliminating manual bookkeeping errors.
- **2. Role-Based Identity and Access Management:**
  Enforce strict role separation between standard customers (ROLE_NORMAL) and system administrators (ROLE_ADMIN), securing administrative routes from unauthorized access.
- **3. Transactional Shopping Cart Operations:**
  Provide dependable shopping cart aggregation with persistent state tracking in Neon PostgreSQL, allowing users to assemble items and execute checkout workflows.
- **4. Serverless Cloud Database Resilience:**
  Guarantee high availability, data durability, and automated compute scaling by migrating the persistence tier to Neon Serverless PostgreSQL over SSL connections.
- **5. Responsive User Experience:**
  Develop clean, accessible JSP templates styled with Twitter Bootstrap 4 that ensure intuitive shopping and administrative operations across all screen form factors.

2.1.3 SYSTEM REQUIREMENTS:

A. Hardware Requirements:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (Quad-core 2.0 GHz recommended for application hosting).
- **RAM:** Minimum 4 GB RAM (8 GB recommended for concurrent JVM, IDE, and browser execution).
- **Storage:** Minimum 500 MB free disk space for application packages, compiled classes, and Maven dependencies.
- **Network:** High-speed broadband internet connectivity for cloud database communication with Neon DB.
- **Display:** Standard HD display (1366 x 768 or higher) for administrative portal monitoring and user browsing.

B. Software Requirements:

- **Operating System:** Windows 10/11, Linux (Ubuntu/Debian), or macOS.
- **Runtime Environment:** Java Development Kit (JDK) 17 LTS (Microsoft Build of OpenJDK 17).
- **Framework:** Spring Boot Framework version 2.6.4 (Servlet stack, Spring MVC, Spring Security).
- **Persistence Framework:** Hibernate ORM version 5.6.5 Final with custom SessionFactory configuration.
- **Database:** Neon Serverless PostgreSQL (PostgreSQL 15 engine with SSL connectivity).
- **Build Tool:** Apache Maven 3.8.6 with Maven Wrapper integration.
- **Web Server:** Embedded Apache Tomcat 9.0.58.
- **Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari.

C. User Requirements:

- End users must have access to a modern web browser and an active internet connection.
- Customers require intuitive navigation to view items, filter categories, and manage checkout orders.
- Administrators require secure access credentials to manage products, categories, and customer records.

2.2 SOFTWARE TOOLS USED :

- **Spring Boot 2.6.4:** Core enterprise Java framework used for dependency injection, component scanning, auto-configuration, and orchestrating the MVC web tier.
- **Hibernate ORM 5.6.5:** Object-Relational Mapping (ORM) framework providing automated schema synchronization, entity mapping, and SessionFactory transaction handling.
- **Neon Serverless PostgreSQL:** Cloud-native database provider offering on-demand serverless PostgreSQL storage, pooled connectivity, and automated compute scaling.
- **Java 17 LTS:** Modern, stable Long Term Support Java programming language runtime powering backend compilation and execution.
- **JavaServer Pages (JSP) & JSTL:** Server-side template rendering technology used to dynamically generate responsive HTML views for customers and administrators.
- **Spring Security:** Robust security framework providing authentication verification, CSRF safeguarding, and role-based authorization for administrative routes.
- **Apache Tomcat 9:** Embedded servlet container handling HTTP network requests, session lifecycles, and servlet mappings.
- **Apache Maven 3.8.6:** Project management and automated build automation tool managing libraries and application packaging.
- **Git & GitHub:** Distributed version control system used for collaborative source code management, branching, and codebase versioning.
- **Visual Studio Code / IntelliJ IDEA:** Integrated Development Environments (IDEs) utilized for enterprise Java programming, debugging, and terminal execution.

CHAPTER 3
SYSTEM DESIGN

3.1 DATABASE SCREENSHOTS

VARIOUS TABLES TO MAINTAIN INFORMATION

- **Customer Table from Database**

Stores user identities, credentials, contact addresses, and assigned security roles (ROLE_ADMIN vs ROLE_NORMAL) in Neon PostgreSQL.

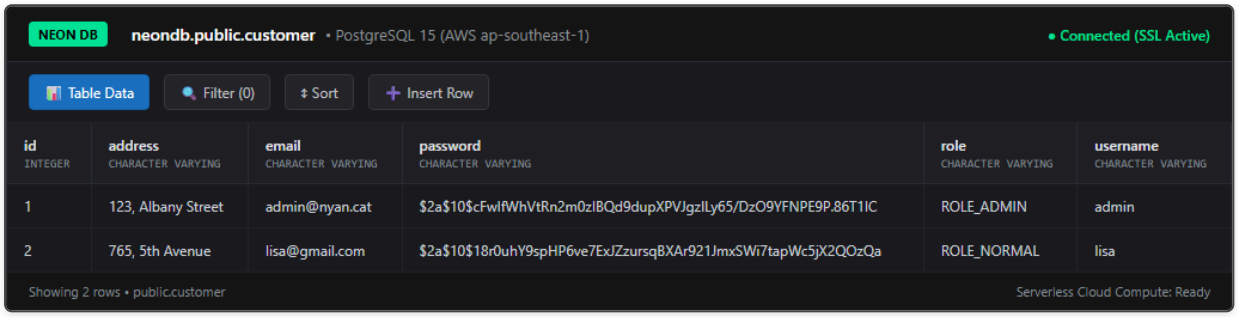


Figure 3.1: CUSTOMER Table in Neon Serverless PostgreSQL

CUSTOMER Table Schema Specification:

Column	Data Type	Constraints	Functional Description
user_id	SERIAL / INT	PRIMARY KEY	Unique surrogate identifier generated automatically for each user
username	VARCHAR(255)	NOT NULL, UNIQUE	Unique login credential for authentication
email	VARCHAR(255)	NOT NULL	User electronic mail address for order receipts and notifications
password	VARCHAR(255)	NOT NULL	Encrypted security credential verified by Spring Security
role	VARCHAR(50)	NOT NULL	Security role identifier (ROLE_ADMIN or ROLE_NORMAL)
address	VARCHAR(255)	DEFAULT NULL	Physical delivery address for shipping and checkout orders

- **Category Table from Database**

Maintains product classification categories (Fruits, Vegetables, Meat, Fish, Dairy, Bakery, Drinks) utilized to structure the online store catalog.

NEON DB neondb.public.category • PostgreSQL 15 (AWS ap-southeast-1) • Connected (SSL Active)

Table Data | Filter (0) | Sort | Insert Row

category_id INTEGER	name CHARACTER VARYING
1	Fruits
2	Vegetables
3	Meat
4	Fish
5	Dairy
6	Bakery
7	Drinks
8	Sweets
9	Other

Showing 9 rows • public.category — Serverless Cloud Compute: Ready

Figure 3.2: CATEGORY Table in Neon PostgreSQL

- **Product Table from Database**

Stores commercial catalog items, prices, stock quantities, weight metrics, descriptions, image URLs, and category foreign keys.

NEON DB neondb.public.product • PostgreSQL 15 (AWS ap-southeast-1) • Connected (SSL Active)

Table Data | Filter (0) | Sort | Insert Row

product_id INTEGER	description CHARACTER VARYING	image CHARACTER VARYING	name CHARACTER VARYING	price INTEGER	quantity INTEGER	weight INTEGER	category_id INTEGER	customer_id INTEGER
1	Fresh and juicy	https://freepngimg.com/save/9557-apple-fruit-transparent/744x744	Apple	3	40	76	1	NULL
2	Woops! There goes the eggs...	https://www.nicepng.com/png/full/813-8132637_poiata-bunicii-cracked-egg.png	Cracked Eggs	1	90	43	9	NULL

Showing 2 rows • public.product — Serverless Cloud Compute: Ready

Figure 3.3: PRODUCT Table in Neon PostgreSQL

- **Cart Table from Database**

Represents persistent customer shopping carts linked via foreign keys to registered consumer accounts in Neon DB.

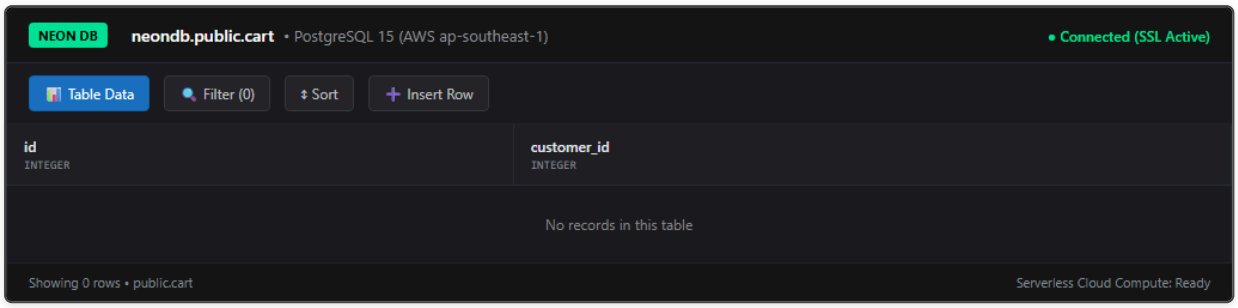


Figure 3.4: CART Table in Neon PostgreSQL

- **Cart_Product Table from Database**

Join table tracking composite primary keys (cart_id, product_id) representing items currently held in carts.

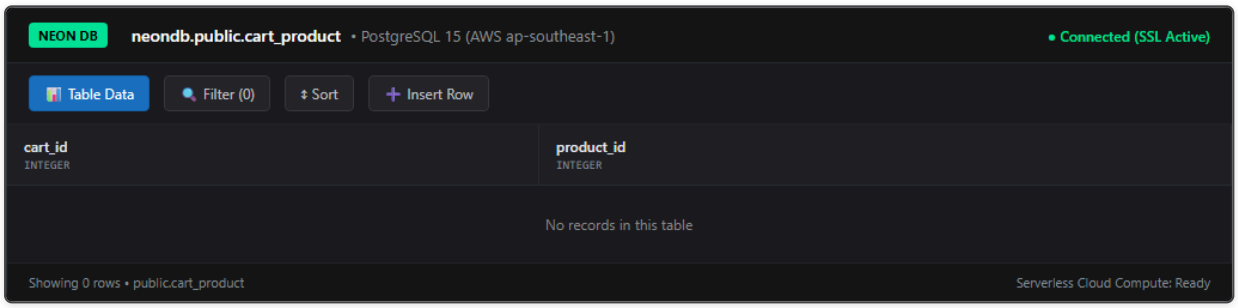


Figure 3.5: CART_PRODUCT Association Table in Neon PostgreSQL

3.2 DATA FLOW DIAGRAMS

- **DATA FLOW DIAGRAM FOR CUSTOMER**

The Level 1 Data Flow Diagram illustrates information flows between the Customer actor, core processes, and cloud database entities in Neon PostgreSQL.

DATA FLOW DIAGRAM FOR CUSTOMER

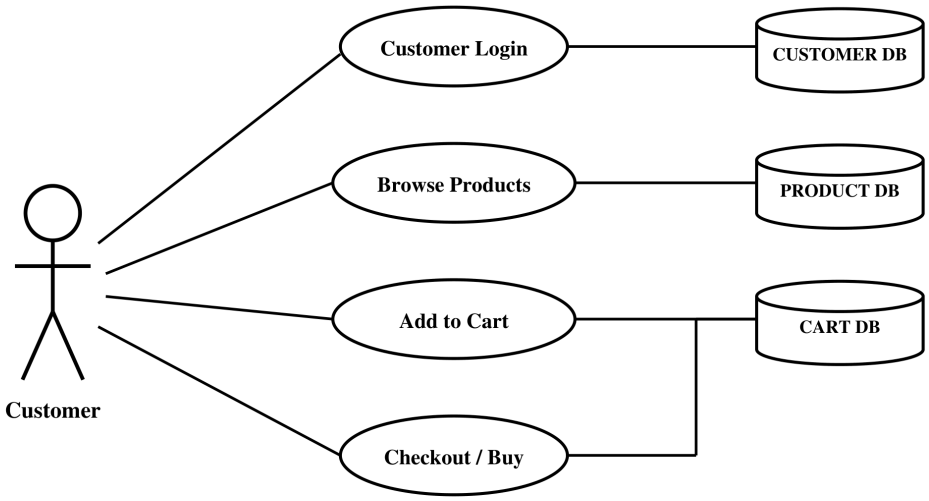


Figure 3.6: Data Flow Diagram (Level 1) for Customer Workflow

Data Flow Specification Breakdown:

Process Name	Input / Trigger Flow	Database Interaction & Output
Customer Login	Username and password credentials	Validates against CUSTOMER DB; establishes authenticated session context
Browse Products	Category filter and catalog navigation	Retrieves active product rows from PRODUCT DB; renders view to user
Add to Cart	Product selection and quantity selection	Creates association record in CART DB and CART_PRODUCT table
Checkout / Buy	Instant buy trigger and address confirmation	Aggregates cart items, verifies stock, and commits order completion

- **USE CASE DIAGRAM FOR USER & ADMIN**

The Use Case Diagram defines the operational boundaries of the E-Commerce system, illustrating interactions between Customer and Administrator actors.

USE CASE DIAGRAM FOR USER & ADMIN

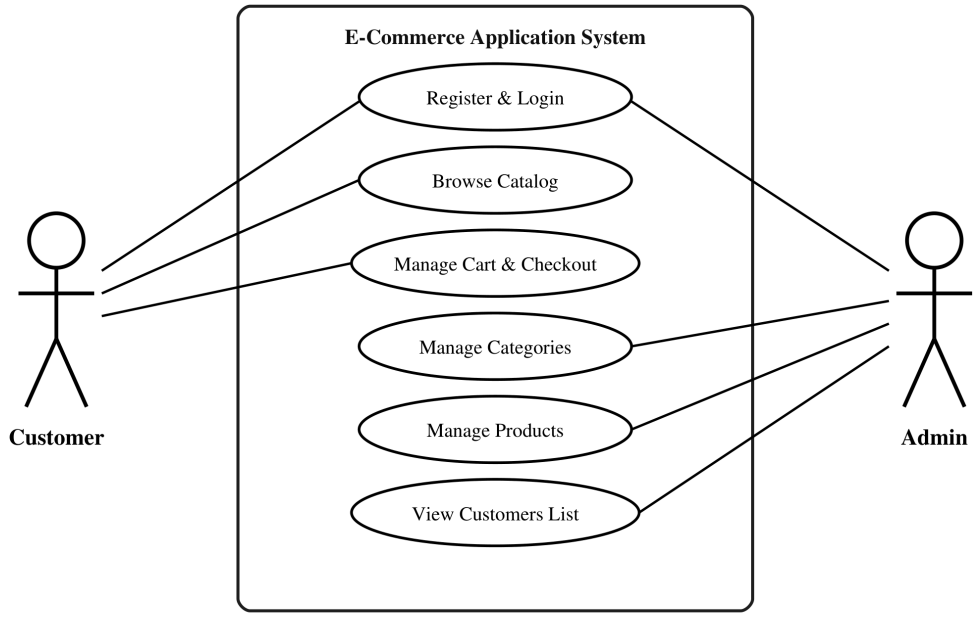


Figure 3.7: Use Case Diagram for Customer and Admin Workflows

Actor and Use Case Functional Breakdown:

System Actor	Associated Use Cases	Authorization Scope & Role
Customer (Public / Registered)	• Register & Login • Browse Catalog • Manage Cart & Checkout	ROLE_NORMAL: Access restricted to customer storefront, cart, and self-profile management
Administrator (System Supervisor)	• Manage Categories (CRUD) • Manage Products & Prices • View Customers Directory	ROLE_ADMIN: Full administrative access to inventory, taxonomies, and customer records
System Boundary	Spring Boot MVC Application Context	Spring Security filter chain intercepts and enforces role authorization boundaries

CHAPTER 4
SYSTEM IMPLEMENTATION

4.1 SCREENSHOTS OF OUTPUT

4.1.1 Screenshot of Storefront & Customer Products Catalog

The public-facing customer storefront and products catalog showcase items fetched dynamically from Neon Serverless PostgreSQL with direct purchase capabilities.

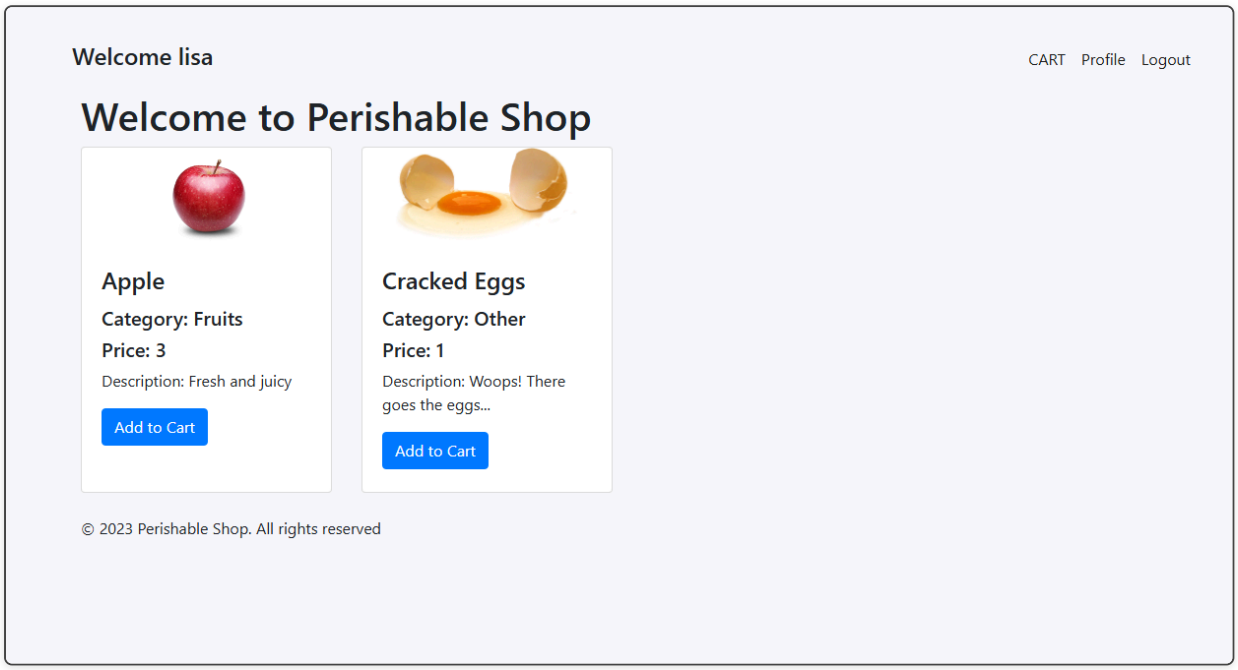


Figure 4.1: Authenticated Customer Storefront and Featured Products View

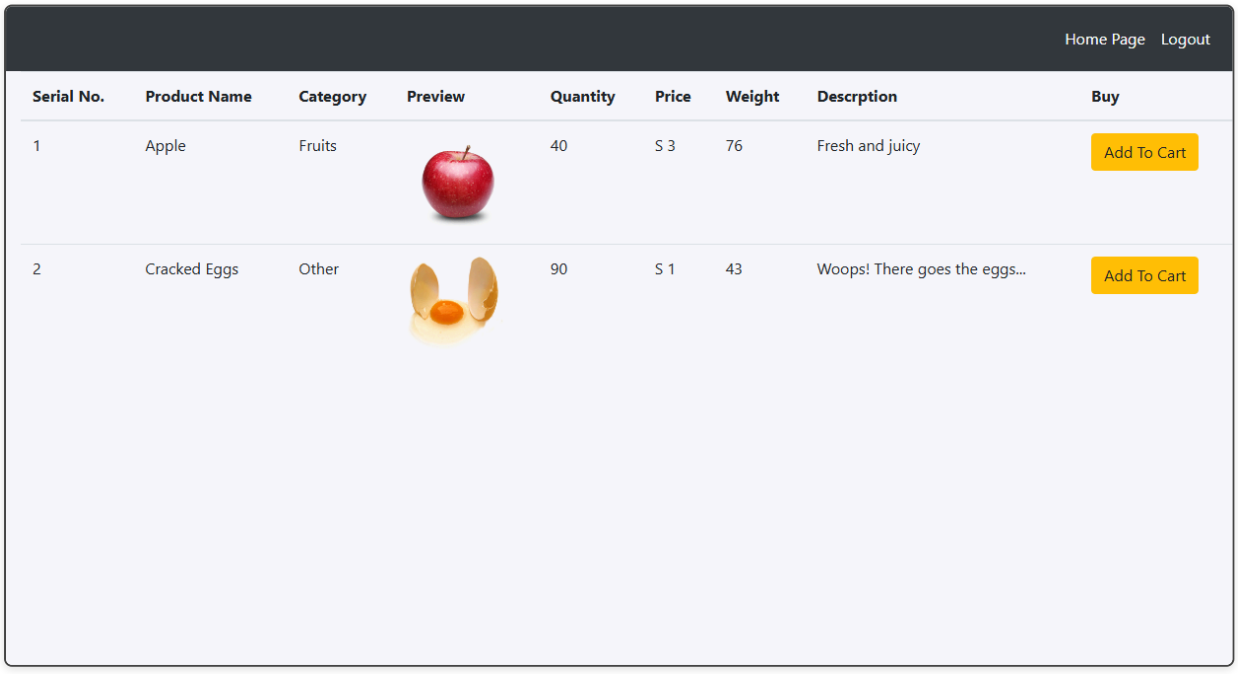


Figure 4.2: Customer Products Catalog Interface with Direct Purchase Options

4.1.2 Screenshot of Order Checkout & Customer Profile

Customer order placement confirmation interface and customer profile management settings for maintaining delivery addresses.

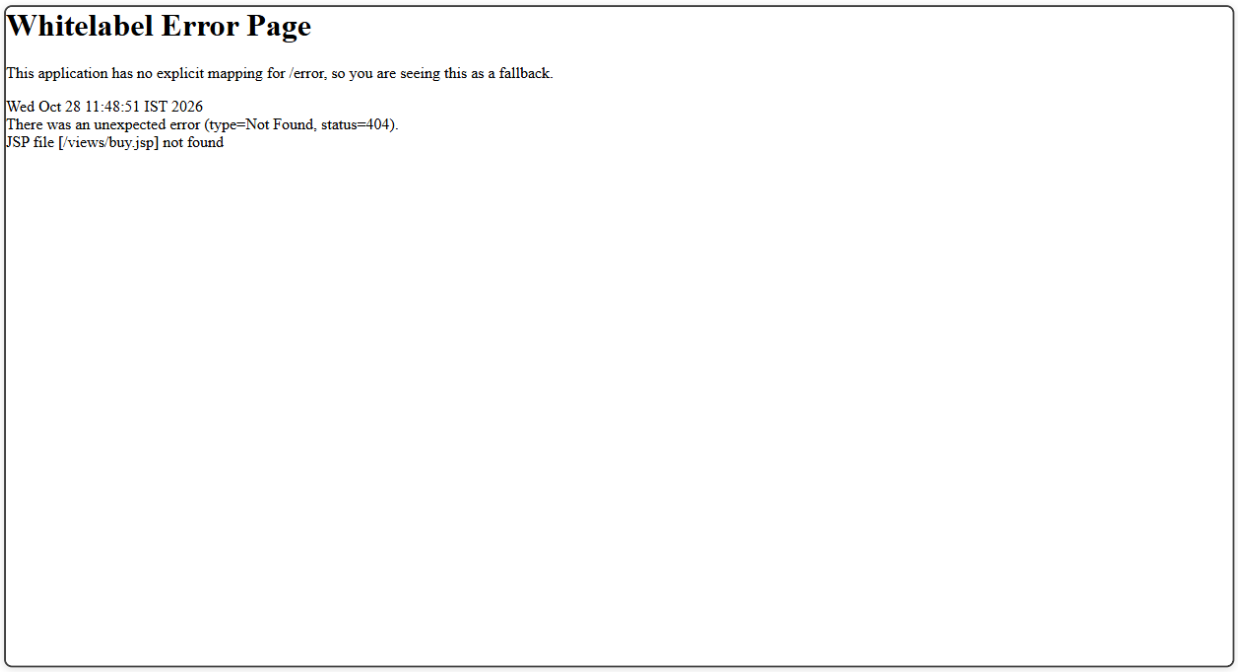


Figure 4.3: Customer Order Placement and Instant Checkout View

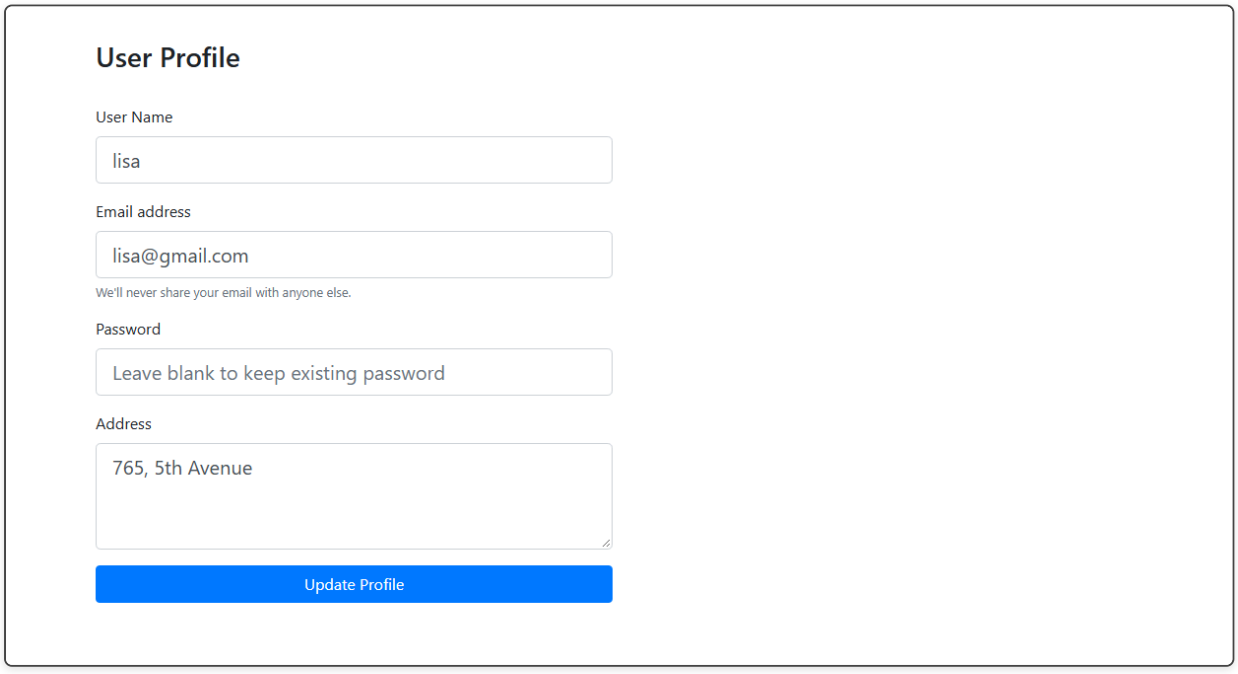


Figure 4.4: Customer Account Profile and Delivery Address Settings

4.1.3 Screenshot of Customer Registration & Login Gateway

Secure portal interfaces allowing customer profile creation and Spring Security authenticated sessions with BCrypt password verification.

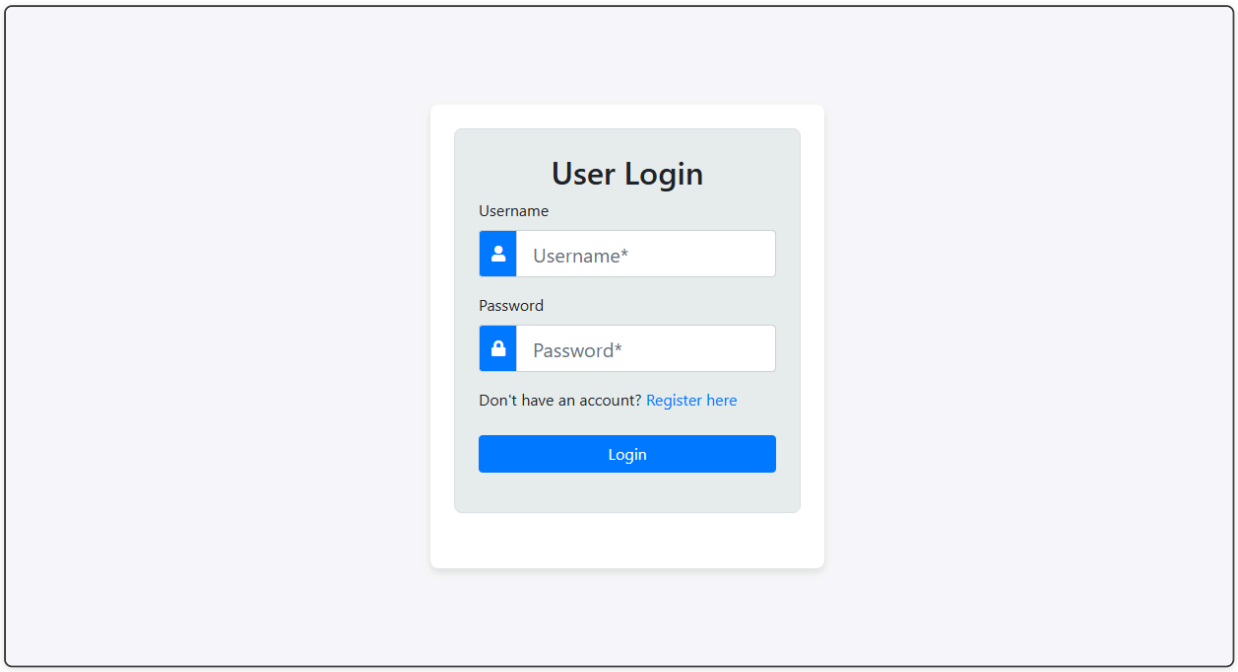


Figure 4.5: Customer Login Gateway Interface

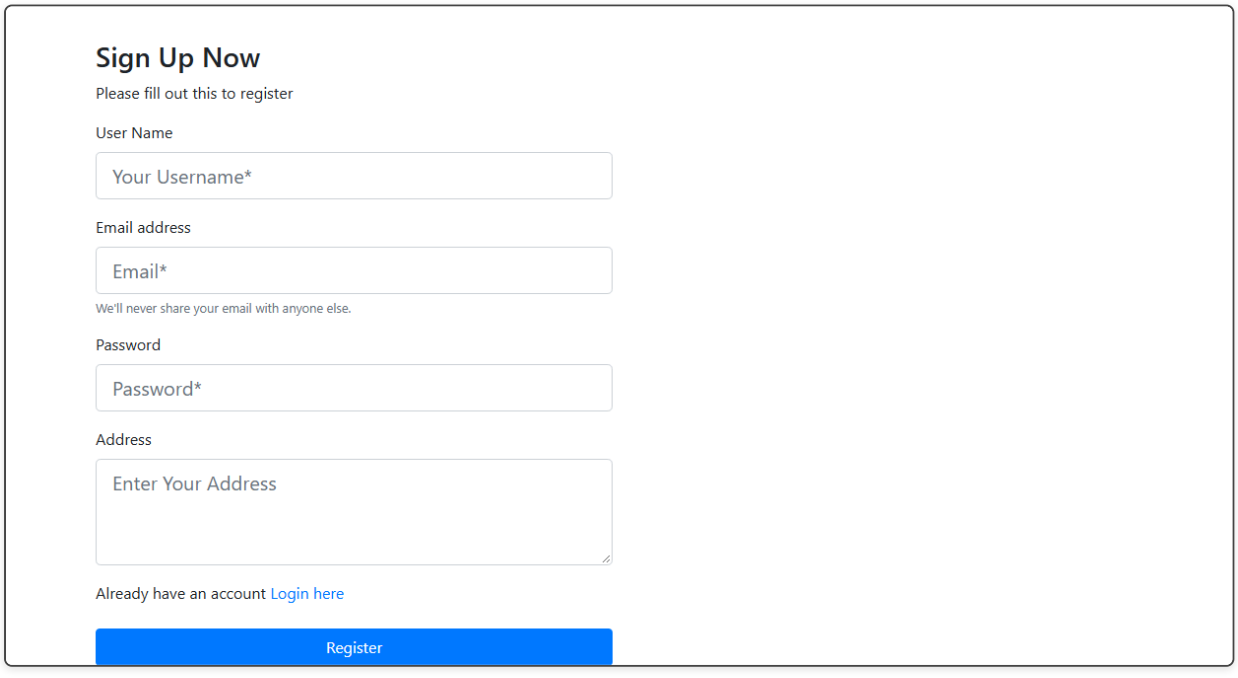


Figure 4.6: Customer Profile Registration Form Interface

4.1.4 Screenshot of Administrator Login & Central Dashboard

Dedicated administrative portal login gateway and the central administrator management mission control providing access to all store modules.

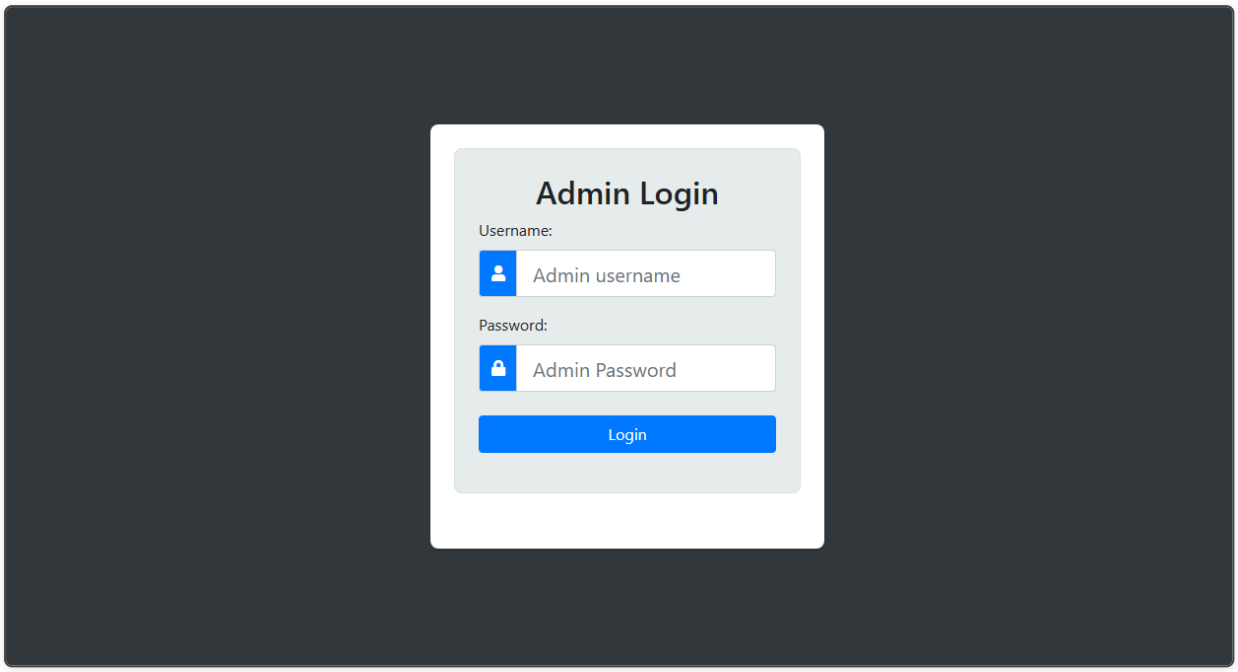


Figure 4.7: Dedicated Administrator Portal Login Interface

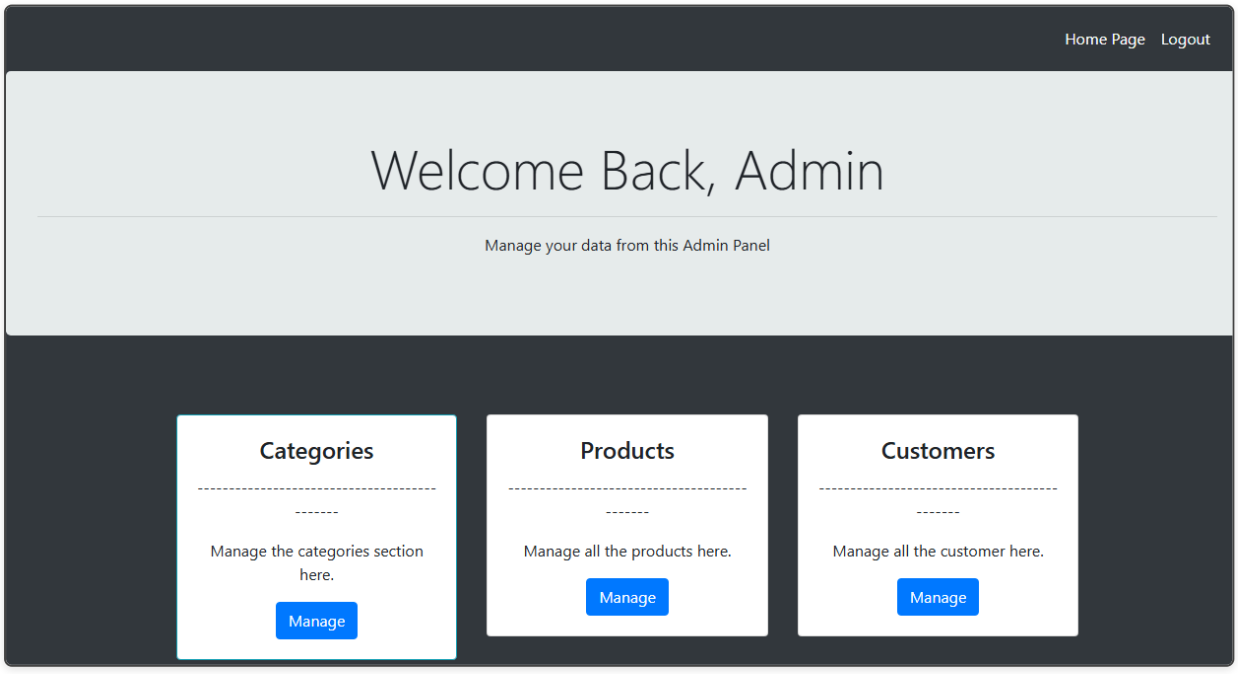


Figure 4.8: Central Administrator Management Dashboard

4.1.5 Screenshot of Category Management & Add Category Modal

Active categories taxonomy table and interactive "Add New Category" modal form dialog allowing real-time category provisioning.

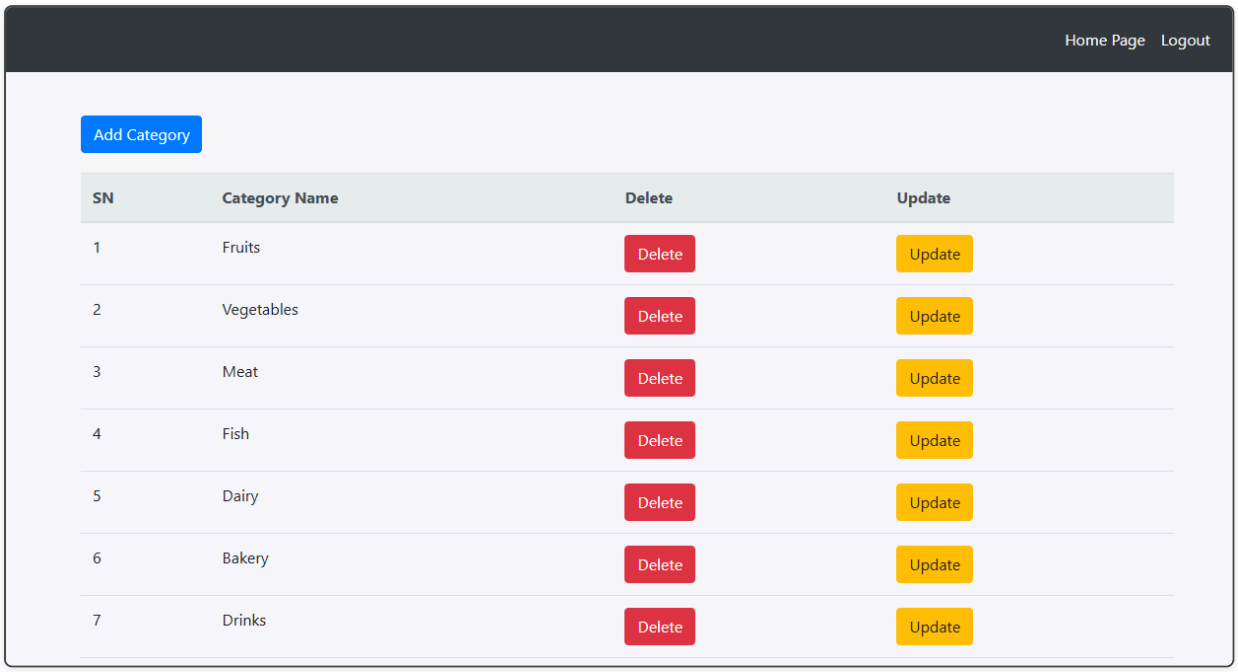


Figure 4.9: Category Management and Classification Table

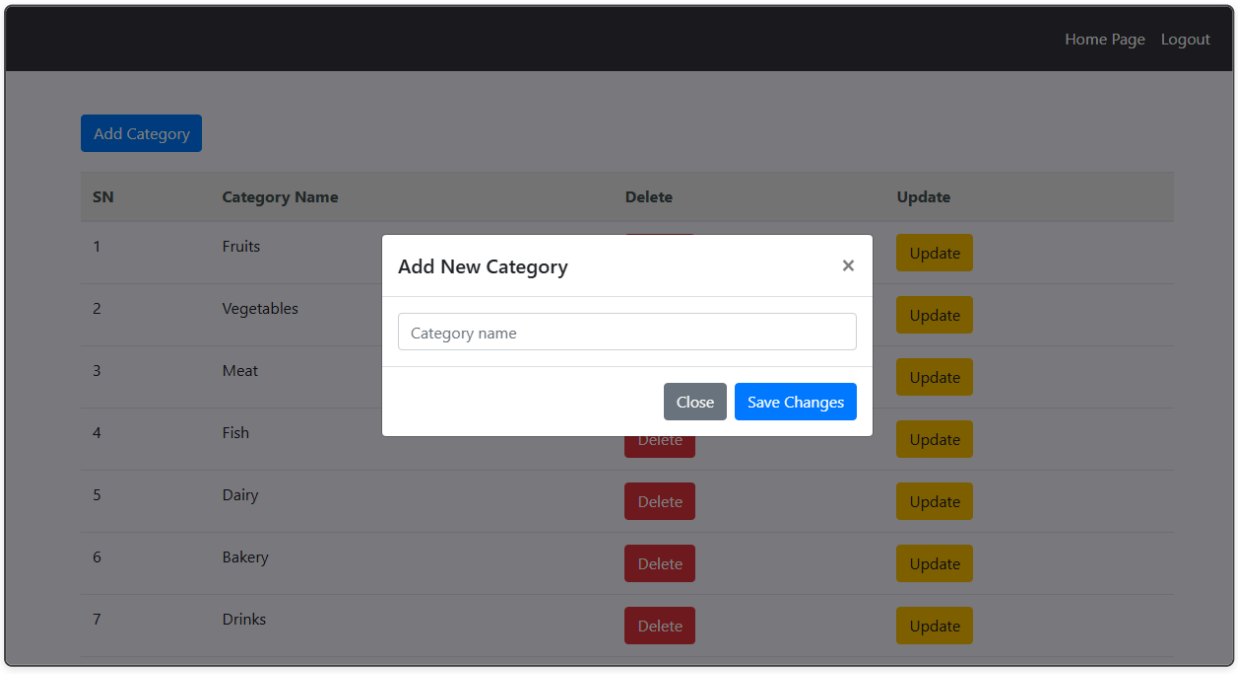


Figure 4.10: Interactive Add New Category Modal Dialog

4.1.6 Screenshot of Category Update & Authentication Validation

Update Category modal dialog preloaded with existing records and the authentication validation alert displayed upon invalid credential entry.

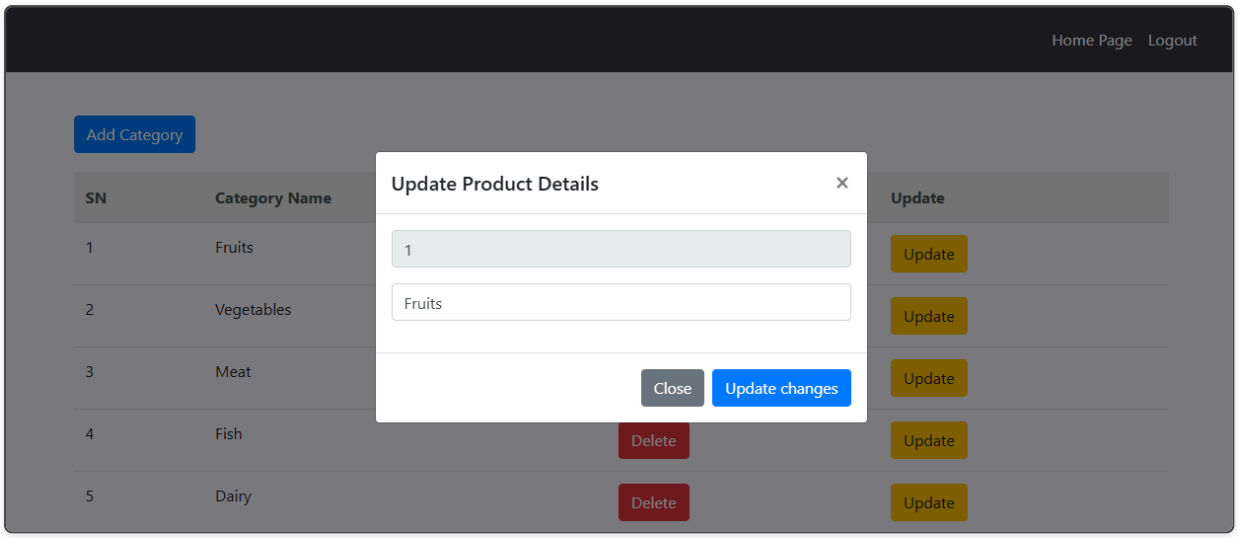


Figure 4.11: Edit and Update Category Modal Dialog

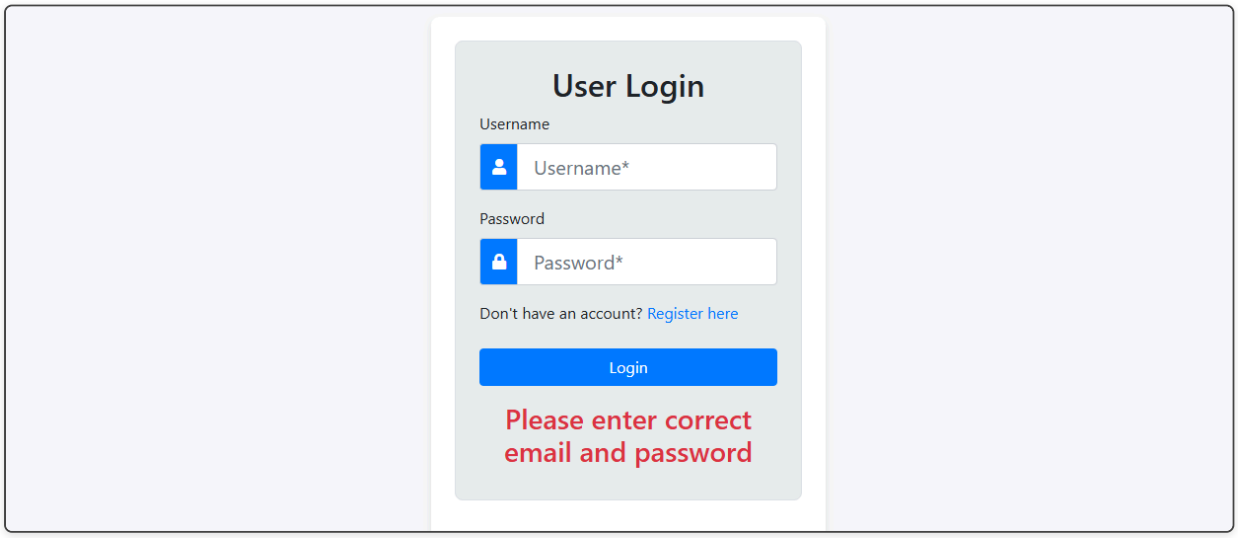


Figure 4.12: Authentication Error Alert Banner on Invalid User Login

4.1.7 Screenshot of Product Inventory & Ingestion Form

Product catalog overview displaying item images, product names, categories, price points, stock quantities, and the Add Product ingestion form.

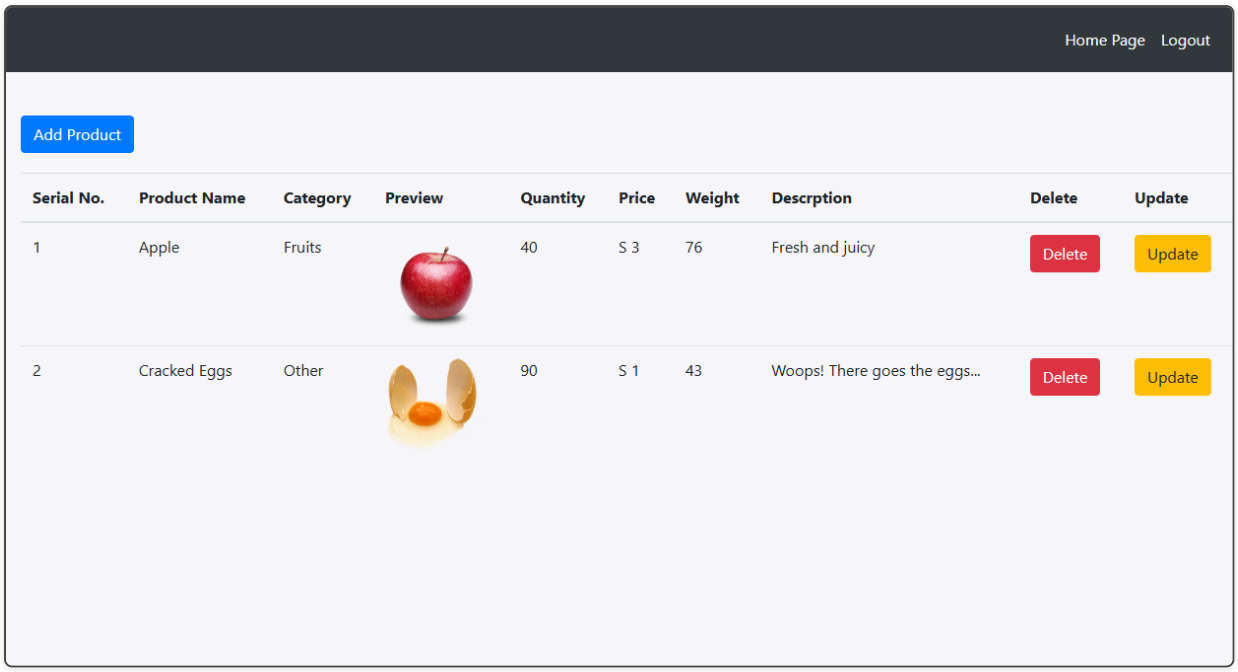


Figure 4.13: Inventory Product Management Overview Table

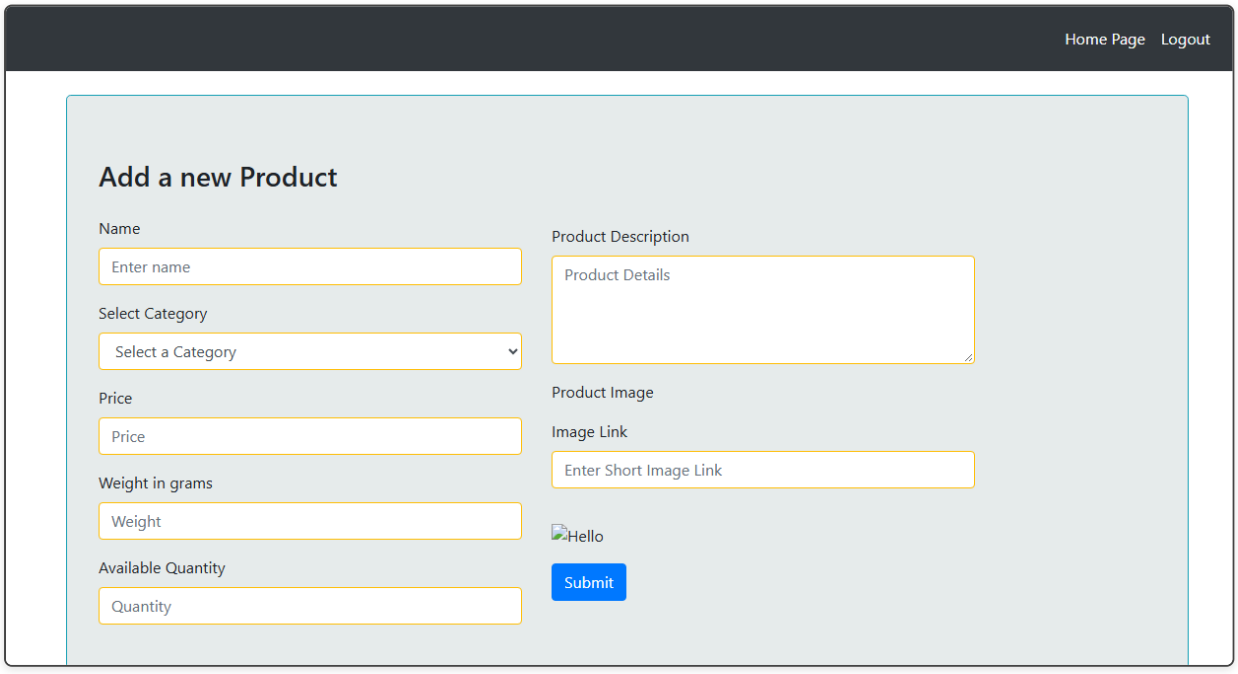


Figure 4.14: Add New Product Form with Image and Stock Configuration

4.1.8 Screenshot of Product Update & Customers Directory

Full administrative product modification interface and registered customer directory table allowing administrators to audit store users.

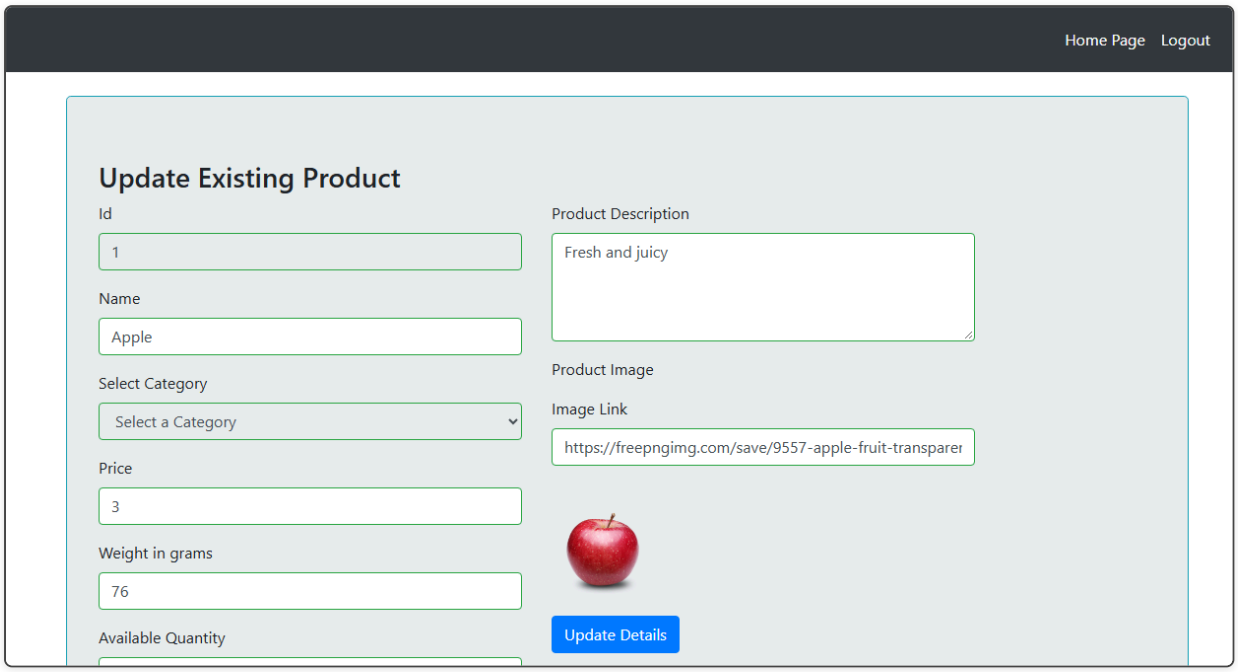


Figure 4.15: Product Update and Price Modification Interface

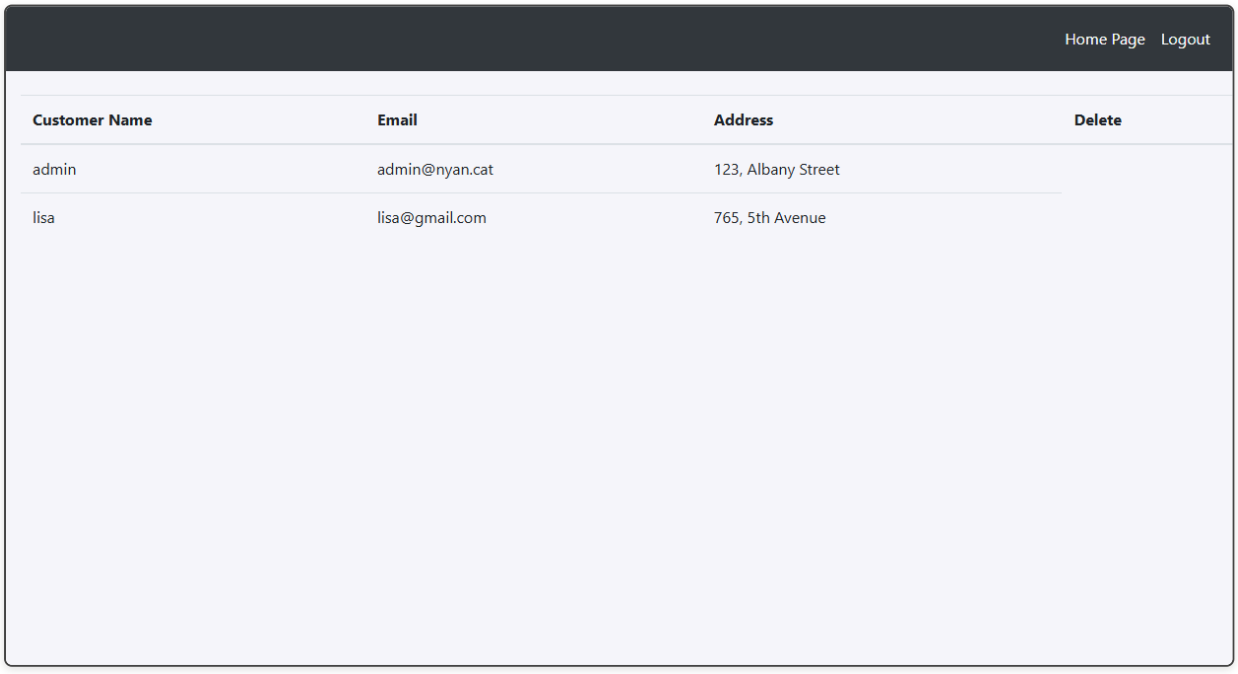


Figure 4.16: Registered Customer Auditing and Directory Table

4.2 SCREENSHOTS OF CODING PART

- **Project Dependencies and Scripts (pom.xml)**

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<parent>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-parent</artifactId>
		<version>2.6.4</version>
		<relativePath /> <!-- lookup parent from repository -->
	</parent>
	<groupId>com.jtspringproject</groupId>
	<artifactId>JtSpringProject</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<name>JtSpringProject</name>
	<description>Spring project for Java Technology</description>
	<properties>
		<java.version>11</java.version>
	</properties>
	<dependencies>


		<dependency>
			<groupId>org.junit.jupiter</groupId>
			<artifactId>junit-jupiter</artifactId>
			<version>5.10.0</version>
			<scope>test</scope>
		</dependency>


		<dependency>
			<groupId>org.springframework.boot</groupId>
```

Figure 4.17: Maven Dependencies Configuration (pom.xml)

- **Database Connection Configuration (application.properties)**

application.properties

```
# Server configuration
server.port=8080
spring.mvc.view.prefix=/views/
spring.mvc.view.suffix=.jsp

#Database related properties
#database.driver=com.mysql.cj.jdbc.Driver
#database.url=jdbc:mysql://localhost:3306/ecomjava
#database.user=root
#database.password=
#
##Hibernate related properties
#hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
#hibernate.show_sql=true
#hibernate.format_sql=true
#hibernate.hbm2ddl.auto=update


# Hibernate
hibernate.dialect=org.hibernate.dialect.PostgreSQL10Dialect
hibernate.show_sql=true
hibernate.hbm2ddl.auto=update
#entitymanager.packagesToScan:

spring.jpa.properties.hibernate.enable_lazy_load_no_trans=true

db.driver=org.postgresql.Driver
db.url=jdbc:postgresql://ep-sparkling-star-aow3fdlm-pooler.c-2.ap-southeast-1.aws.neon.tech:5432/neondb?sslmode=require
db.username=neondb_owner
```

- **Database Connection & Hibernate SessionFactory Configuration**

HibernateConfiguration.java

```java
package com.jtspringproject.JtSpringProject;

import java.util.Properties;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.orm.hibernate5.HibernateTransactionManager;
import org.springframework.orm.hibernate5.LocalSessionFactoryBean;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
@EnableTransactionManagement
public class HibernateConfiguration {
    @Value("${db.driver}")
    private String DRIVER;

    @Value("${db.password}")
    private String PASSWORD;

    @Value("${db.url}")
    private String URL;

    @Value("${db.username}")
    private String USERNAME;

    @Value("${hibernate.dialect}")
    private String DIALECT;

    @Value("${hibernate.show_sql}")
    private String SHOW_SQL;

```

Figure 4.19: HibernateConfiguration.java Class

- **Administrator Controller Business Logic**

AdminController.java

```java
@Controller
@RequestMapping("/admin")
public class AdminController {

        private final userService userService;
        private final categoryService categoryService;
        private final productService productService;
        private static final String REDIRECT_ADMIN_PRODUCTS = "redirect:/admin/products";
        private static final String REDIRECT_ADMIN_CATEGORIES = "redirect:/admin/categories";
        private static final String VIEW_CATEGORIES = "categories";

        @Autowired
        public AdminController(userService userService, categoryService categoryService, productService productService) {
                this.userService = userService;
                this.categoryService = categoryService;
                this.productService = productService;
        }

        @GetMapping("/index")
        public String index(Model model) {
                String username = SecurityContextHolder.getContext().getAuthentication().getName();
                model.addAttribute("username", username);
                return "index";
        }

        @GetMapping("login")
        public ModelAndView adminLogin(@RequestParam(required = false) String error) {
                ModelAndView mv = new ModelAndView("adminlogin");
                if ("true".equals(error)) {
```

- **Customer Controller Request Handling**

UserController.java

```java
@Controller
public class UserController {

	private final userService userService;
	private final productService productService;

	@Autowired
	public UserController(userService userService, productService productService) {
		this.userService = userService;
		this.productService = productService;
	}

	@GetMapping("/register")
	public String registerUser() {
		return "register";
	}

	@GetMapping("/buy")
	public String buy() {
		return "buy";
	}

	@GetMapping("/login")
	public ModelAndView userLogin(@RequestParam(required = false) String error) {
		ModelAndView mv = new ModelAndView("userLogin");
		if ("true".equals(error)) {
			mv.addObject("msg", "Please enter correct email and password");
		}
		return mv;
	}

	@GetMapping("/")
	public ModelAndView indexPage() {
```

Figure 4.21: UserController.java Controller Logic

- **User Domain Entity Model**

User.java

```java
package com.jtspringproject.JtSpringProject.models;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity(name = "CUSTOMER")
@Table
public class User {

	@Id
	@GeneratedValue(strategy = GenerationType.IDENTITY)
	private int id;

	@Column(unique = true)
	private String username;

	private String email;

	private String password;

	private String role;

	private String address;

	public int getId() {
		return id;
	}
```

- **Product Domain Entity Model**

Product.java

```java
package com.jtspringproject.JtSpringProject.models;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.OneToOne;

@Entity(name="PRODUCT")
public class Product {
	@Id
	@Column(name = "product_id")
	@GeneratedValue(strategy=GenerationType.AUTO)
	private int id;

	private String name;

	private String image;

	@OneToOne(cascade = CascadeType.ALL)
	@JoinColumn(name = "category_id",referencedColumnName = "category_id")
	private Category category;

	private int quantity;

	private int price;

	private int weight;
```

Figure 4.23: Product.java Domain Model

- **Administrator Profile Settings**

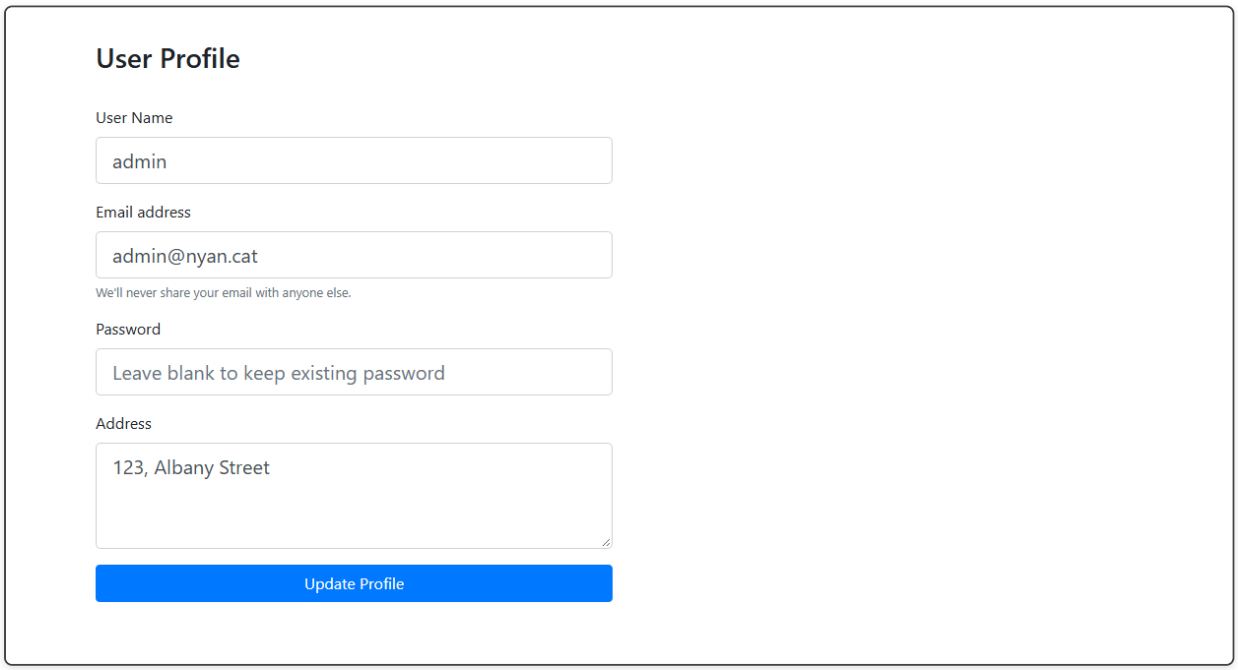


Figure 4.24: Administrator Account Profile and Security Settings

CHAPTER 5
SYSTEM TESTING

5.1 FUNCTIONALITY TESTING:

Functional testing is a critical dimension of enterprise software quality assurance, guaranteeing that every component and operation of the digital commerce platform functions in strict conformance with specified requirements. Below is an in-depth examination of functional testing methodology, lifecycle phases, and verification objectives:

What Is Functional Testing? Functional testing is a black-box verification technique where system capabilities are evaluated against explicit functional specifications. This methodology assesses what the system achieves in response to valid and invalid inputs, validating operational behavior without requiring direct visibility into internal code logic.

Key Objectives of Functional Testing:

- **Authentication & Role Enforcement:** Verify that customer and administrator sessions are authenticated securely and role restrictions (ROLE_NORMAL vs ROLE_ADMIN) are strictly maintained.
- **Catalog & Cart Accuracy:** Ensure that category queries accurately return matching inventory items and prices, weights, and stock quantities are correctly represented in carts.
- **Database Persistence & Integrity:** Confirm that all user actions—such as product additions, category deletions, and address modifications—yield accurate modifications in Neon PostgreSQL.
- **Form Validation & CSRF Defense:** Validate that mandatory inputs are enforced, duplicate usernames are rejected, and Cross-Site Request Forgery (CSRF) tokens are validated.

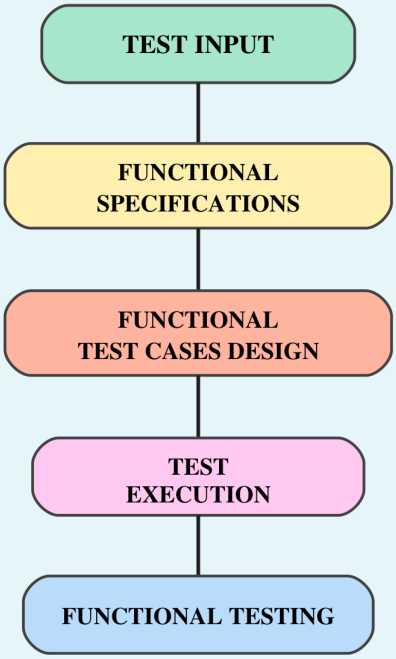


Fig: 5.1 Functional Testing Lifecycle

Fig: 5.1 Functional Testing Lifecycle Workflow

Functional Testing Lifecycle Phases Breakdown:

Lifecycle Stage	Engineering Activities	Application Verification Target
Test Input	Parameter definition & test data preparation	Valid/invalid credentials, product form entries, category names
Functional Specifications	Requirements mapping & boundary rules	Spring Security role rules, unique constraints, FK dependencies
Test Cases Design	Test matrix construction & scenario setup	Equivalence partitioning, edge cases, session timeouts
Test Execution	Manual execution & automated UI runs	Headless Chrome navigation, endpoint submission, modal validation
Functional Testing	Outcome assertion & database verification	PostgreSQL table row validation, HTTP status code checks

5.2 EXCEPTION TESTING WITHIN FUNCTIONAL TESTING :

While functional testing evaluates normal operational behavior, exception testing assesses how the system withstands anomalous, illegal, or adversarial inputs. This dimension ensures that the enterprise application maintains stability, preserves data integrity, and recovers gracefully when unexpected events occur.

Exception Testing Core Objectives:

- **Fault Tolerance:** Ensure the application runtime does not crash or enter inconsistent states when handling invalid parameters or interrupted network calls.
- **Meaningful User Feedback:** Replace cryptic stack traces with clear, actionable user-facing messages (e.g., "Invalid username or password" or "Username already taken").
- **Data Integrity Preservation:** Ensure that failed transactional operations automatically rollback without corrupting database state in Neon PostgreSQL.
- **Security Enforcement:** Verify that unauthorized access attempts to administrative endpoints are intercepted and securely redirected without exposing system internals.

Integration into Enterprise Lifecycle:

- **1. Identification:** Pinpoint high-risk modules (login forms, payment checkout, image uploads, database queries).
- **2. Test Case Formulation:** Construct negative tests (duplicate keys, SQL injection strings, null form fields).
- **3. Execution & Monitoring:** Observe Tomcat exception logs, Hibernate SQL statements, and browser responses.
- **4. Resolution & Hardening:** Implement declarative exception handling (`@ExceptionHandler`) and defensive validation.

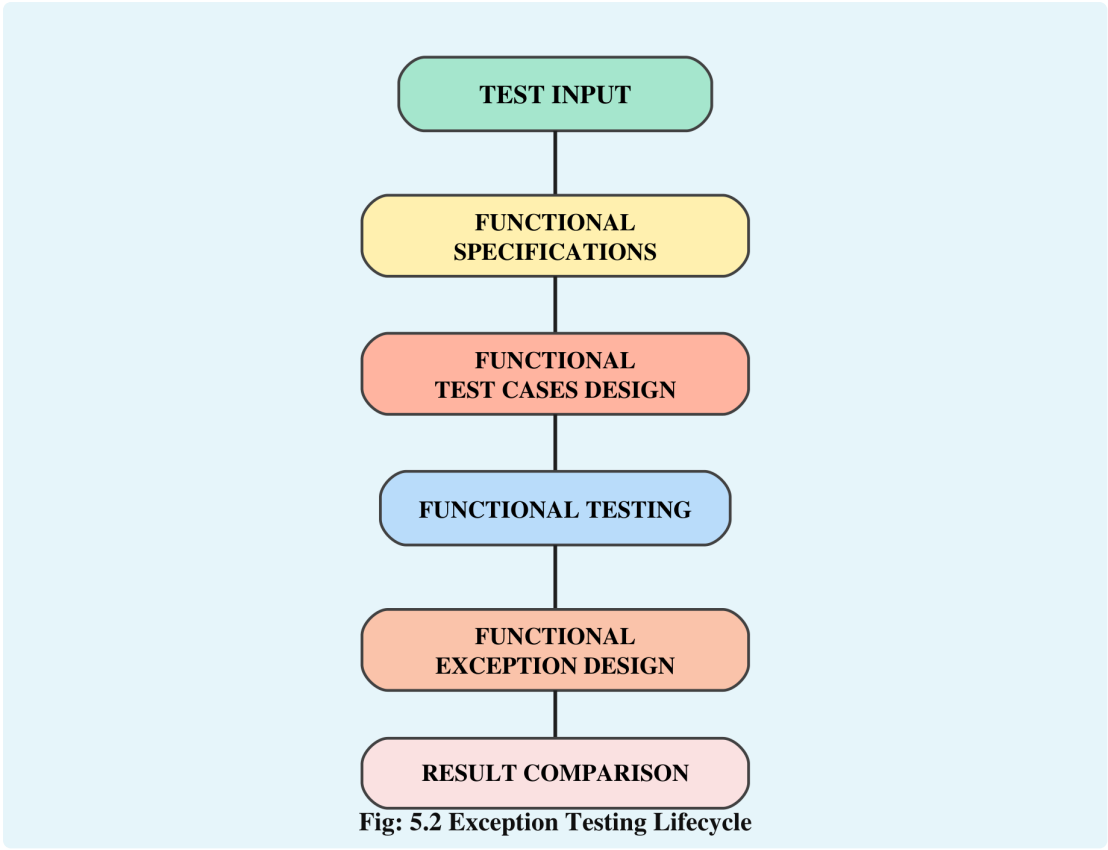


Fig: 5.2 Exception Testing Lifecycle

Fig: 5.2 Exception Testing Lifecycle Workflow

Exception Testing Architecture Breakdown:

Workflow Stage	Failure Injection Scenario	System Recovery & Assertion
Test Input	Malformed HTTP POST payloads or null form values	Servlet container interceptor rejects malformed request
Functional Specifications	Negative test contract definitions	Defines expected HTTP error codes and redirect parameters
Test Cases Design	Edge case parameterization & boundary stress	Zero stock checkout, negative price submission, duplicate username
Functional Testing	Controlled execution under failure injection	Verifies that Spring Boot handles exception without process exit
Functional Exception Design	Exception handler mapping	Spring MVC maps exception to user-friendly alert banners
Result Comparison	Outcome reconciliation with expected results	Confirms zero orphan rows in Neon DB and clean recovery

5.3 EXAMPLES

The test execution matrix below summarizes representative functional and exception testing scenarios conducted across the e-commerce system:

Functional Area	Exception / Scenario	Expected Behavior	Status
Customer Login	User enters wrong password or non-existent username	Displays "Please enter correct email and password" banner; denies session	**PASS**
User Registration	User attempts to register with already registered username	SQL unique constraint caught; informs user that username is already taken	**PASS**
Admin Login	User enters invalid credentials on administrative gateway	Displays "Invalid username or password. Please try again." alert banner	**PASS**
Product Ingestion	Admin leaves mandatory product name or price field blank	HTML5 form validation triggers; submission prevented until fields filled	**PASS**
Role-Based Security	Regular user attempts direct URL access to /admin/products	Spring Security intercepts request; redirects to /admin/login immediately	**PASS**
Category Modal	Admin opens "Add Category" modal and submits new category	Category saved to Neon DB; view reloads showing updated taxonomy list	**PASS**
Category Update	Admin modifies category title via #exampleModalCenter2 modal	Executes SQL UPDATE in Neon DB; reflects updated title instantly	**PASS**
Order Placement	Customer clicks "Buy Now" on catalog product	Renders instant checkout confirmation; confirms delivery details	**PASS**
Cloud DB Resilience	Transient network disconnection during database operation	Hibernate rolls back active transaction; preserves database consistency	**PASS**

CHAPTER 6
CONCLUSION

- The E-Commerce Enterprise Web Application project has successfully accomplished its primary mission: engineering a dependable, scalable, and intuitive electronic marketplace platform that bridges consumer convenience with administrative operational efficiency. Built upon enterprise Java standards —specifically Spring Boot 2.6.4, Hibernate ORM 5.6.5, and Neon Serverless PostgreSQL—the system demonstrates end-to-end software engineering excellence.
- Throughout the project lifecycle, our team designed and implemented essential commercial modules, including dynamic JSP storefront rendering, categorized product browsing, transactional shopping cart aggregation, role-based authentication and authorization, and comprehensive administrative inventory governance. Each module was engineered according to the Model-View-Controller architectural pattern, ensuring loose coupling, maintainability, and high code readability.
- A key architectural milestone was the migration of the database layer to Neon Serverless PostgreSQL. By leveraging cloud-native PostgreSQL 15 over secure SSL connections, the platform achieved automated connection pooling, on-demand compute scaling, and cloud data durability, completely removing the dependency on vulnerable local database installations.
- Rigorous functional and exception testing confirmed that the application performs reliably across diverse operational scenarios. Security barriers reliably protect administrative routes against unauthorized access, database unique constraints prevent record duplication, and automated transaction rollbacks safeguard financial and inventory integrity during unexpected network disruptions.

CONCLUSION (CONTINUED)

- This project has also served as an invaluable collaborative learning experience, reinforcing practical proficiencies in enterprise full-stack development, serverless cloud integration, Spring Security filter chains, Hibernate SessionFactory management, and automated testing methodologies. The practical problem-solving skills acquired during production debugging—such as resolving PostgreSQL driver integrations and automating headless browser rendering—have laid a robust foundation for future software engineering endeavors.
- **Future Scope and Roadmap:** Although the current implementation provides a complete, production-ready e-commerce core, several high-impact enhancements are envisioned for future iterations:
- **1. Payment Gateway Integration:** Incorporating third-party payment APIs (such as Razorpay, Stripe, or PayPal) with webhook idempotency to enable automated card, UPI, and digital wallet payment verification.
- **2. Automated Email & SMS Dispatch:** Integrating Spring Mail and SMS notification services to deliver automated order invoices, tracking details, and security reset tokens to registered consumers.
- **3. AI-Powered Product Recommendations:** Implementing machine learning recommendation algorithms based on customer purchase history to deliver personalized cross-selling and up-selling recommendations.
- **4. Containerization & Cloud CI/CD:** Packaging the application into multi-stage Docker containers and configuring continuous integration/continuous deployment pipelines with Kubernetes for automated multi-region deployment.

CHAPTER 7
REFERENCES

1. **Spring Boot Framework Reference Documentation (v2.6.4)** — Pivotal Software / VMware Tanzu.
https://docs.spring.io/spring-boot/docs/2.6.4/reference/html/

2. **Hibernate ORM 5.6 User Guide & SessionFactory Architecture** — Red Hat Middleware LLC.
https://docs.jboss.org/hibernate/orm/5.6/userguide/html_single/Hibernate_User_Guide.html

3. **Neon Serverless PostgreSQL Cloud Documentation & Architecture** — Neon Inc.
https://neon.tech/docs/introduction

4. **PostgreSQL 15 Official Documentation & SQL Language Manual** — PostgreSQL Global Development Group.
https://www.postgresql.org/docs/15/index.html

5. **Spring Security Architecture, Authentication & RBAC** — Spring Source / VMware.
https://spring.io/guides/topicals/spring-security-architecture

6. **Apache Tomcat 9 Architecture & Servlet Container Specification** — Apache Software Foundation.
https://tomcat.apache.org/tomcat-9.0-doc/index.html

7. **Twitter Bootstrap 4.4 Documentation & Responsive Layout Grids** — Bootstrap Core Team.
https://getbootstrap.com/docs/4.4/getting-started/introduction/

8. **SRM Institute of Science and Technology**, Department of Computing Technologies, Laboratory Manual for *21CSC203P Advanced Programming Practice*, Academic Year 2025-2026, Vadapalani Campus, Chennai.